

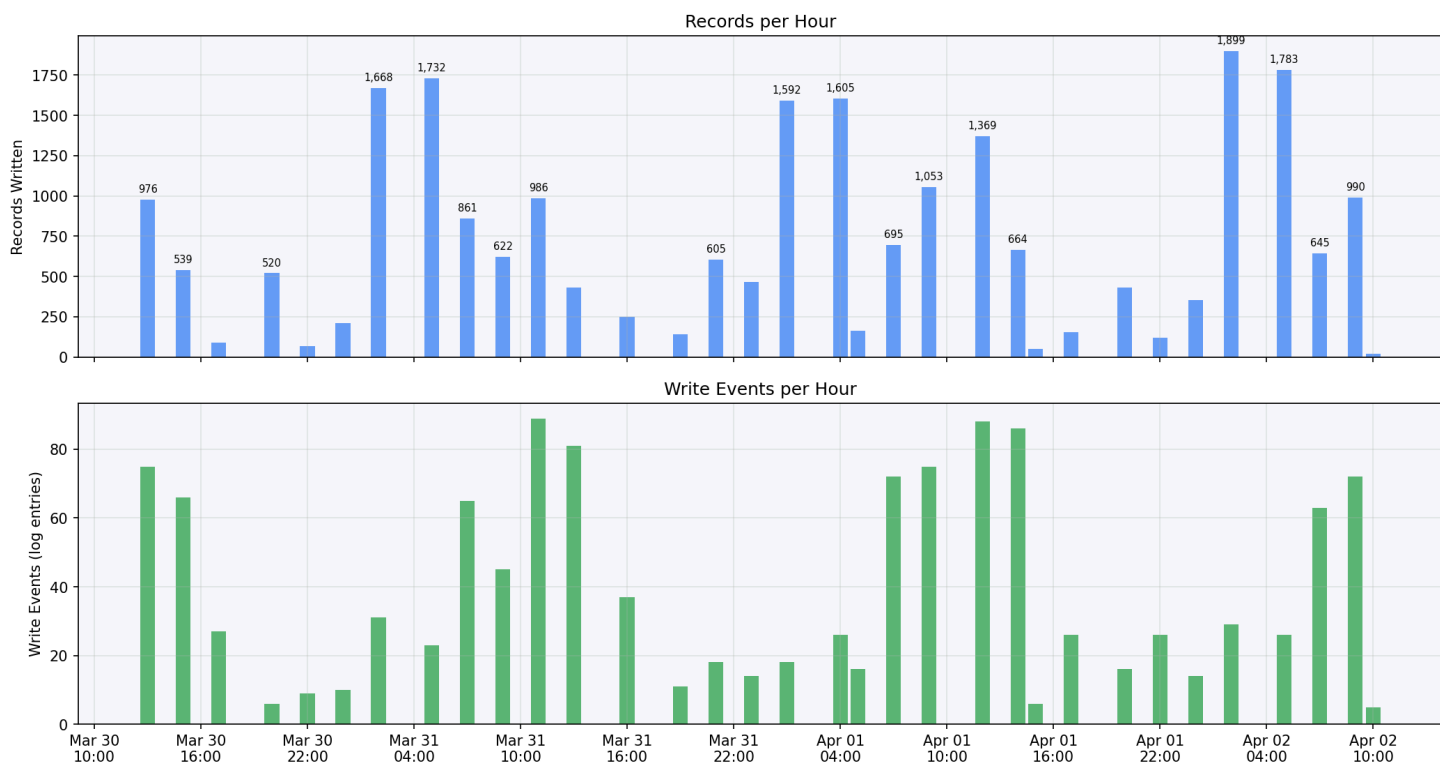
Datastream CDC Analysis Report

Stream: orauat-1060-bucket
Source: Oracle UAT (TMS1060_SENDUNG)
Target: gs://tms-alloydb-datastream-bucket-wl5-t-t/UATDataStream
Period: 2026-03-30 13:00 — 2026-04-02 12:00

Throughput

Metric	Value
Write events (log entries)	1,271
Total records written	23,751
Errors	0
Delivery rate	100.00%
Batch size (min / avg / max)	1 / 19 / 1,098

Datastream CDC Throughput — orauat-1060-bucket 2026-03-30 13:00 — 2026-04-02 12:00



End-to-End Latency (DB change to GCS object)

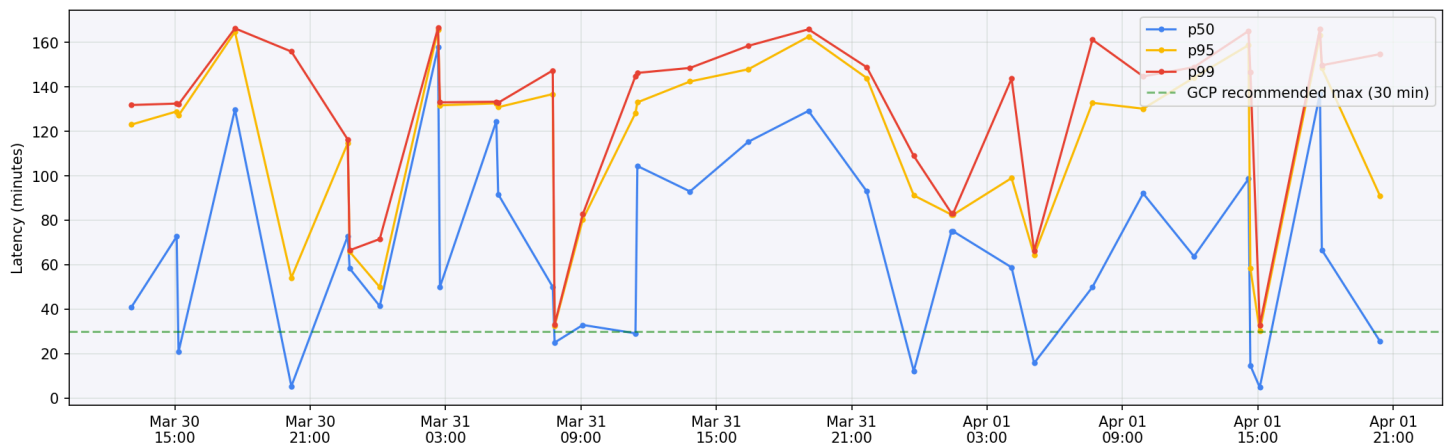
Metric: datastream.googleapis.com/stream/total_latencies

Official definition: Time from when data is written to the source until the corresponding events are written to the destination.

Source: [Monitor a stream | GCP Docs](#)

Percentile	Min	Avg	Max
P50	4.8 min	66.5 min	157.8 min
P95	30.3 min	113.5 min	165.8 min
P99	32.7 min	127.7 min	166.5 min

End-to-End Latency (Total Latencies) — DB change to GCS object
orauat-1060-bucket



System Latency (Datastream processing)

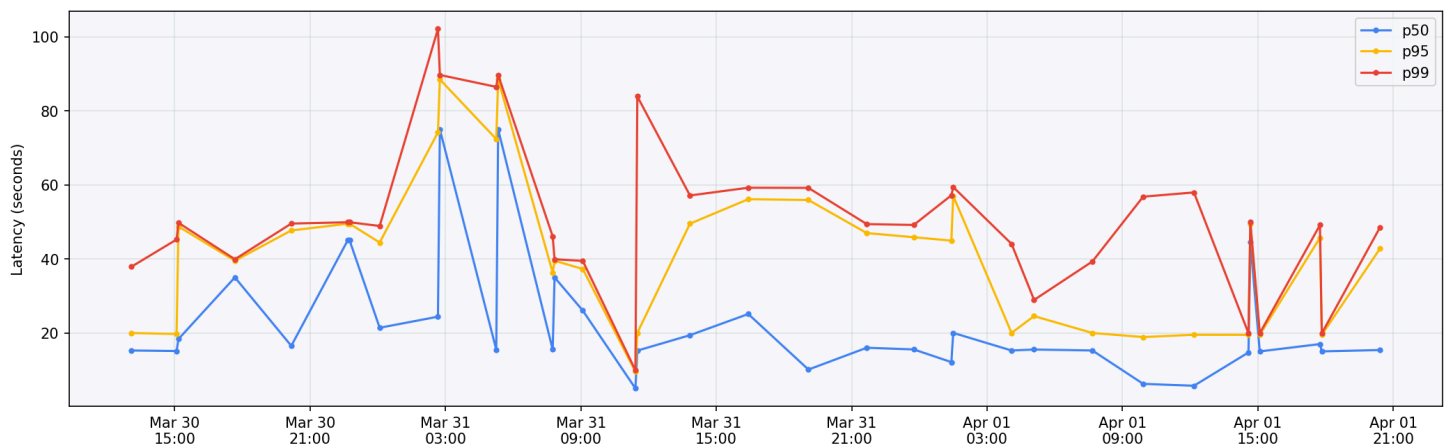
Metric: datastream.googleapis.com/stream/system_latencies

Official definition: Time from when Datastream reads the event until it writes to the destination.

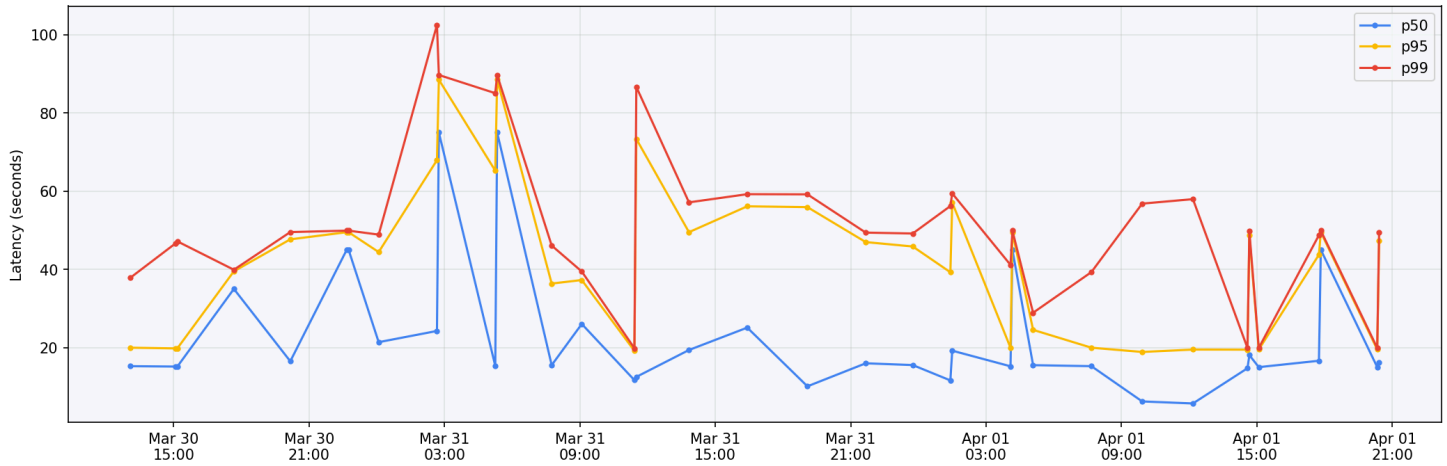
Source: [Monitor a stream | GCP Docs](#)

Percentile	Min	Avg	Max
P50	5.0s	22.7s	75.0s
P95	9.5s	41.2s	88.5s
P99	9.9s	50.9s	102.1s

System Latency (Datastream Processing) — read to write
orauat-1060-bucket



Stream Latency — orauat-1060-bucket



Stream Freshness

Metric: `datastream.googleapis.com/stream/freshness`

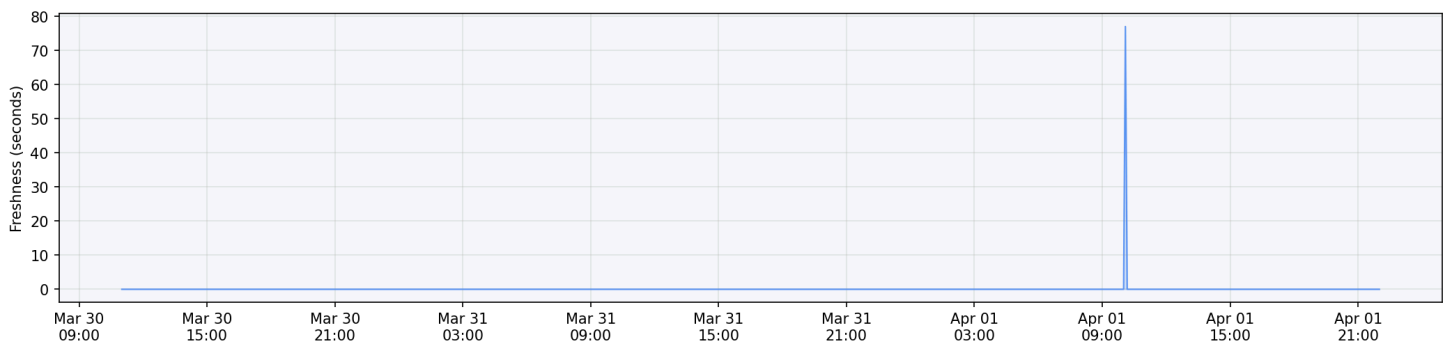
Official definition: Difference between when data was committed to the source and when Datastream reads it. Set to 0 if there are no new events to read.

Formula: $\text{Total Latency} = \text{Freshness} + \text{System Latency}$

Source: [Monitor a stream | GCP Docs](#)

709 data points, **almost entirely 0** — the stream is keeping up with the source whenever it polls.

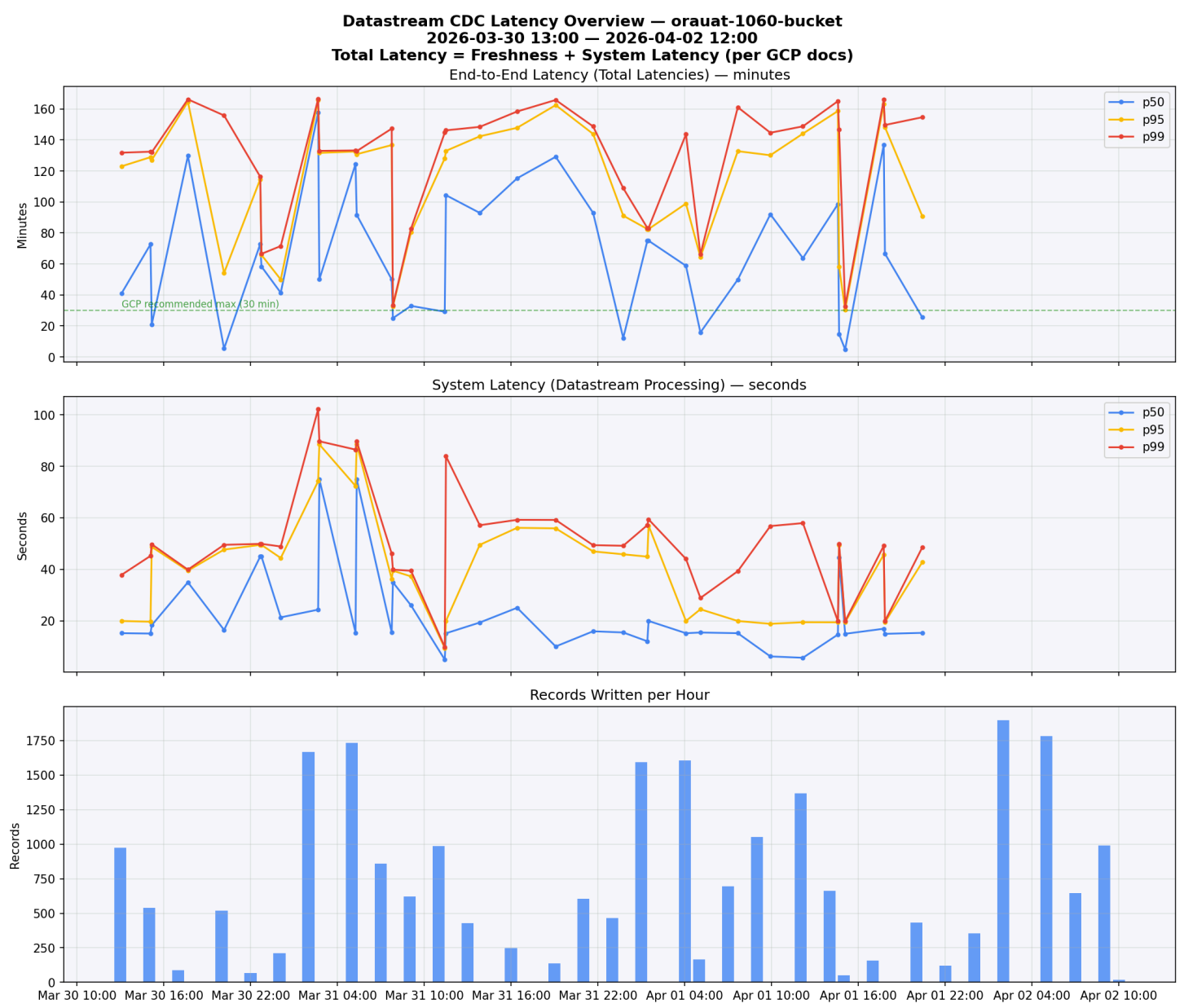
Stream Freshness — time between source commit and Datastream read
orauat-1060-bucket



Latency Breakdown

Component	Value	% of total
Total end-to-end (p50 avg)	66.5 min	100%
Datastream processing (p50 avg)	23s	0.6%
Read lag / queue time	~66 min	99.4%

99.4% of the end-to-end latency is NOT Datastream processing — it is the time between the database change occurring and Datastream reading it from Oracle's archived redo logs via LogMiner.



Key Factors Affecting Read Lag

Per GCP documentation, the following factors determine how quickly Datastream can read changes from Oracle:

1. **Oracle redo log archival frequency** (primary factor)

- Datastream reads from *archived* redo logs, not online logs
- Minimum latency = redo log switch interval
- Recommended: switch every 10-20 minutes, log size < 256MB
- Current avg latency of ~66 min suggests switches may be ~60 min apart

2. **LogMiner is single-threaded**

- Subject to higher latency with high transaction volumes
- Alternative: Binary Log Reader (Preview) — multithreaded, reads online + archived logs

3. **maxConcurrentCdcTasks** parameter

- Tune in Datastream to read more logs in parallel during peak hours

4. **Oracle source health**

- CPU, SGA/PGA sizing, disk I/O, Streams Pool Size
- Source overload: when logs are generated faster than Datastream can read them

5. **Oracle 19c deprecated CONTINUOUS_MINE**

- Before 19c, LogMiner could read online redo logs in real-time
- Now archived-log-only, adding inherent latency

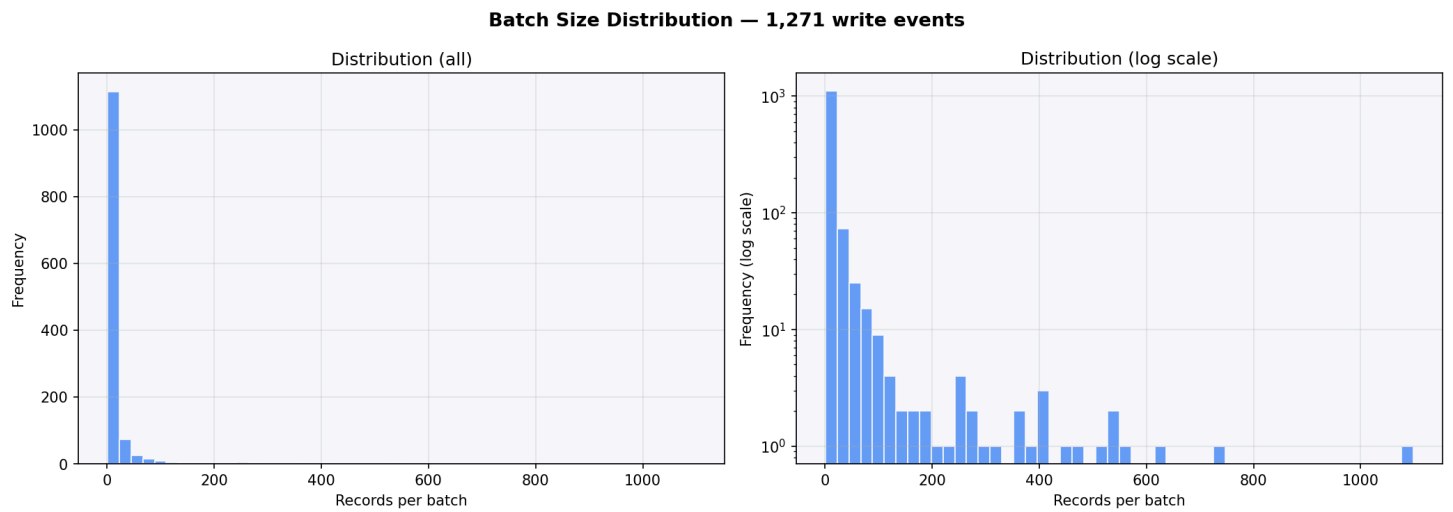
GCP expected latency: up to 30 min end-to-end. Our observed avg of 66 min suggests the redo log switch frequency on the Oracle source should be optimized.

Batch Size Distribution

Batch size	Entries	Records	% of total records
1	213	213	0.9%
2-5	497	1,515	6.4%
6-10	215	1,651	7.0%
11-50	271	5,482	23.1%
51-100	37	2,615	11.0%

Batch size	Entries	Records	% of total records
101-500	31	7,668	32.3%
500+	7	4,607	19.4%

Large batches (500+) occur overnight (~02:00-05:00) — accumulated changes flushed in bulk during low-activity hours.



Datastream vs. Striim — Why Striim is Faster

The observed avg end-to-end latency of ~66 min with GCP Datastream raises the question: why does Striim achieve sub-second latency on the same Oracle source?

Root Cause: How They Read Oracle Redo Logs

	GCP Datastream	Striim
Read method	LogMiner, archived redo logs only	Reads online redo logs directly (+ archived)
Mining mode	Waits for log archival before reading	Active Log Mining (ALM) — reads in real-time
Alternative reader	Binary Log Reader (Preview, not GA)	OJet — reads Oracle LCRs directly via API

	GCP Datastream	Striim
Throughput	Not published	Oracle Reader: 20-80 GB/hr, OJet: 150+ GB/hr
Latency	~30-60+ min (depends on log rotation)	Sub-second

GCP Datastream waits for Oracle to archive the redo log before it can read changes. This means the minimum latency equals the redo log switch interval (typically 15-60 min). **Striim reads from the online redo log in real-time** — it does not wait for the archive step.

Analogy:

- **Datastream** = waits for the newspaper to be printed and delivered
- **Striim** = reads the journalist's screen as they type

Oracle 19c Deprecation of CONTINUOUS_MINE — Impact on All Three Approaches

Oracle 19c deprecated `CONTINUOUS_MINE`, a LogMiner API feature that previously allowed reading online redo logs in real-time. **None of the current CDC solutions rely on it** — they all work around it differently:

Solution	Uses CONTINUOUS_MINE?	Workaround
Datastream LogMiner (GA)	No	Only reads archived logs — accepts the latency
Datastream Binary Log Reader (Preview)	No	Bypasses LogMiner entirely — parses raw binary redo files directly
Striim ALM	No	Own implementation reading online redo logs without CONTINUOUS_MINE
Striim OJet	No	Bypasses LogMiner entirely — reads Oracle LCRs via API

Datastream Binary Log Reader (Preview) — The Comparable Option

GCP's Binary Log Reader is conceptually similar to Striim's approach:

- Reads **online + archived** redo log files (not just archived)
- **Multithreaded** (not single-threaded like LogMiner)
- **Bypasses LogMiner** — parses raw binary redo files directly via ASM or database directory objects
- Supports **low-latency CDC**

Limitations vs. Striim:

- **Preview / Pre-GA** — not production-ready, limited support
- **Reduced data type coverage** compared to LogMiner method
- Does not support Oracle 11g (12c+ only)
- Still recommends tuning `maxConcurrentCdcTasks` and log switch frequency

Implication for This POC

The ~66 min avg latency observed in this Datastream POC is **inherent to the LogMiner (GA) method's architecture** (archived-log-only mode), not a configuration issue. Options to reduce latency:

1. **Tune redo log rotation** — aim for 10-20 min switches to approach the GCP-stated ~30 min target
2. **Switch to Binary Log Reader (Preview)** — would significantly reduce latency by reading online logs, but is not yet GA
3. **Use Striim** — sub-second latency, production-ready, but adds a third-party dependency and cost

Documentation Sources

GCP Datastream

- [Monitor a stream | Datastream](#) — official metric definitions
- [General best practices | Datastream](#) — latency tuning guidance

- [Work with Oracle redo log files | Datastream](#) — redo log configuration
- [Stream data from Oracle databases | Datastream](#) — LogMiner vs Binary Log Reader comparison
- [Configure a self-managed Oracle database | Datastream](#) — Binary Log Reader setup
- [End-to-end latency for Oracle to BigQuery | Google Cloud Community](#) — deep dive on latency components
- [Life After LogMiner: Oracle CDC with Datastream | ExpertBeacon](#) — LogMiner limitations and alternatives

Striim

- [Striim Oracle Database CDC readers](#) — Oracle Reader vs OJet comparison
- [Striim: Oracle CDC Methods, Benefits, Challenges](#) — architecture overview
- [Striim: LogMiner Continuous Mining Deprecation](#) — ALM and OJet as replacements
- [Google Cloud Blog: Oracle to BigQuery with Striim](#) — GCP + Striim integration

Output Files

File	Contents
consolidated_report.csv	Hourly breakdown: events, records, all latencies
consolidated_summary.json	Machine-readable full summary
consolidated_report.md	This file